

← Go Back

Hardware

03. Uploading Sketches to the M4 Core on Arduino Portenta X8
04. Data Exchange Between Python® on Linux & Arduino Sketch
05. Managing Containers with Docker on Portenta X8
06. Using FoundriesFactory® Waves Fleet Management
07. Deploy a Custom Container with Portenta X8 Manager
08. How To Build a Custom Image for Your Portenta X8
09. How To Update Your Portenta X8
10. Data Logging with MQTT, Node-RED, InfluxDB and Grafana
11. Output WebGL Content on a Screen
12. Multi-Protocol Gateway With Portenta X8 & Max Carrier
13. Running WordPress & Database Containers on Portenta X8
14. Portenta X8 Firmware Release Notes
15. Edge AI Flow Monitoring on Portenta X8 with Docker
16. Getting Started with ROS 2 and Turtlesim on Portenta X8

Home / Hardware / Portenta X8 / 10. Data Logging with MQTT, Node-RED, InfluxDB and Grafana

10. Data Logging with MQTT, Node-RED, InfluxDB and Grafana

This tutorial will show you how to set up a local data logging application using an MQTT broker, Node-RED, InfluxDB, Grafana, and the Arduino® Portenta X8.

Author

José Bagur and Taddy Chung

Last revision

09/25/2024

ON THIS PAGE

Overview

- Goals

+
- IoT Architecture Basics
- Installing Mosquitto

+
- Installing Node-RED

+
- Installing InfluxDB

+
- Installing Grafana

+
- Sending Data Using the MKR WiFi 1010 Board
- Conclusion

+

Overview

This tutorial will set up a typical Internet of Things (IoT) application: an MQTT data logger. For this application, we are going to use what we call in the Arduino team the "IoT-Quartet", four common building blocks that are popular and regularly used in these types of applications:

- Mosquitto (MQTT broker)
- Node-RED
- InfluxDB
- Grafana

These four blocks will be running

Portenta X8 board. We will use data from an **Arduino® MKR WiFi 1010** board to test the data logging application.

Goals

Install, configure, and run Mosquitto (MQTT broker) locally in the Portenta X8

Install, configure, and run Node-RED locally in the Portenta X8

Install, configure, and run InfluxDB locally in the Portenta X8

Install, configure, and run Grafana locally in the Portenta X8

Send data from an Arduino® MKR WiFi 1010 board to the data logging application running locally in the Portenta X8

Required Hardware and Software

Arduino® Portenta X8

Arduino® MKR WiFi 1010

USB-C® cable (USB-C® to USB-A cable)

Wi-Fi® Access Point (AP) with Internet access

ADB or SSH

Command-line interface

Arduino IDE 2

If you are new to the Portenta X8 board, check out this **User Manual** on controlling your board using a terminal or



IoT Architecture

Basics

IoT applications and devices are everywhere nowadays, even in places where we don't realize a device or application is connected to the Internet.

Rather than a single technology, **IoT is a concept** that refers to the connection of everyday devices, just like your watch, to the Internet and how that Internet connection creates more and different ways to interact with your device and your environment. Interactions between humans and devices or applications create **data** (often in large quantities) that must be communicated and processed.

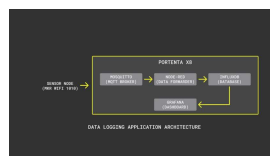
How can we plan, build and deploy IoT solutions? To answer that question, we must consider **IoT architecture**. Due to the variety of IoT devices and applications that exist and can be developed, there is no single, unique architecture for IoT devices and applications. However, we can discuss a foundational architecture that can be considered a starting point for any IoT project. This foundational architecture consists of **three essential layers: perception** (or devices), **network**, and **application**. Let's explore these layers in more detail:

layer, data is gathered by one or more sensor nodes. Actuators, which respond to data collected from sensor nodes, are also part of this layer.

Network layer: This layer is responsible for collecting and transmitting data from sensor nodes to back-end services, such as databases. Data transmission can occur through various communication networks like Wi-Fi or cellular networks.

Application layer: This layer is what the device or application user sees and interacts with, such as a dashboard. The sensor data is processed and displayed to the end-user in a user-friendly manner through smartphone apps or web interfaces.

These three layers are interconnected to form a complete IoT system. In this tutorial, we are going to take this base architecture and set up a data-logging application, as shown in the image below:



IoT application high-level architecture.

In the high-level architecture described in the image above:

The perception layer consists of an MKR WiFi 1010 board; this board will

The network layer consists of the MQTT broker (Mosquitto), the data forwarder (Node-RED), and the database (InfluxDB).

The application layer consists of a dashboard (Grafana), where information from the sensor node is shown.

For documentation purposes, we will explain, step by step, the installation process of each part of the

 application (Mosquitto, Node-RED, InfluxDB, and Grafana); this process can be done quickly using a unique Compose `YAML` file.

Let's start by configuring the MQTT broker!

Installing Mosquitto

Let's start by creating a new directory in our Portenta X8 `home` directory called `mqtt`; inside this directory, we are going to make a file named `docker-compose.yml` using the following commands:

```
1 mkdir mqtt
```

```
1 cd mqtt
```

```
1 export TERM=xterm
```

```
1 stty rows 36 cols 150
```

```
1 sudo vi docker-compose
```

The

```
export TERM=xterm
```

and



```
stty rows 36 cols  
150
```

commands enable VI
editor full screen.

Inside the VI editor, copy and
paste the following:

```
1  services:
2      mqtt:
3          conta
4          image
5          resta
6          user:
7          ports
8
9
10         volum
11
12
13
14  volumes:
15      config:
16      data:
17      log:
```

`1883` is a standard MQTT port; the `8883`

i port is usually used for TLS-secured MQTT connections.

Save the file and exit the VI editor. Return to the `mqtt` directory and run the following command:

```
1  docker-compose up -d
```

The Mosquitto broker should be available on your Portenta X8 `IP address`. You can retrieve the `IP Address` of your board with the `ping <hostname>` command.

We should see inside the `mqtt` directory three folders (`config`, `data`, and `log`) and the `docker-compose.yml` file we

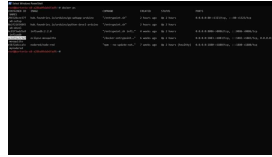
the `docker-compose up -d` command, the logs will fill with errors about being unable to open the configuration file. To fix this issue, we can download a default configuration file and store it in the newly created `config` directory:

```
1 sudo wget https://raw.
```

Let's add the following lines at the end of the `config` file

```
1 # Listen on port 1883
2 listener 1883
3 listener 9001
4
5 # Save the in-memory
6 persistence true
7 persistence_location
8
9 # Log to stderr and I
10 log_dest stderr
11 log_dest file /mosqui
```

Save the file and exit the VI editor. Now, let's restart the Mosquitto container so the configuration file can start working. This can be done using the `docker restart` command and the Mosquitto `CONTAINER ID`. To identify the Mosquitto `CONTAINER ID`, run the `docker ps` command and copy the `CONTAINER ID`, as shown in the image below:



Docker container ID.

Now, we need to manage password files by adding a user to a new password file. For this, we need to run the `sh` command in the mosquitto container with the mosquitto `CONTAINER ID` found before, as shown below:

```
1 docker exec -it CONTAINER ID sh
```

Let's dissect that command:

`docker exec` runs a command in a running container (in this case, the Mosquitto container)

`-it CONTAINER ID sh` attaches a terminal session into the running container so we can see what is going on with the container and interact with it

Now, in the terminal session with the Mosquitto container, run the following command:

```
1 mosquitto_passwd -c /m
```

This command creates a new password file (

`guest` is the username. After entering the `username` we want, we must define a password for the username and then exit the terminal session with the `exit` command. Now, let's return to the `config` directory; you should see now inside this directory the `mosquitto.passwd` file. Open the `mosquitto.config` file and add the following information to it:

```
1 password_file /mosquitto/passwd
2 allow_anonymous true
```

The file should see now like this:

```
1 # Password file
2 password_file /mosquitto/passwd
3 allow_anonymous true
4
5 # Listen on port 1883
6 listener 1883
7 listener 9001
8
9 # Save the in-memory
10 persistence true
11 persistence_location /mosquitto/data
12
13 # Log to stderr and file
14 log_dest stderr
15 log_dest file /mosquitto/log
```

To test it, save the file and exit the VI editor; also, we need to restart the Mosquitto container so the configuration file can start working. This can be done by

`CONTAINER ID`. After restarting the container, the local Mosquitto broker should be ready.

Testing Mosquitto

To test the Mosquitto broker, we need an MQTT client. We can use several ways to implement an MQTT client, one of the easiest ways is to install an MQTT client in our web browser and use it to test the connection between the local MQTT broker on the Portenta X8 board and the web-based MQTT client. This tutorial will use [MQTTBox](#), a Google Chrome extension that works as an MQTT client.

In MQTTBox, let's start by configuring the settings of the MQTT client. The information we are going to need is the following:

MQTT client name: in this example, Portenta X8 MQTT Broker

Protocol: mqtt/tcp

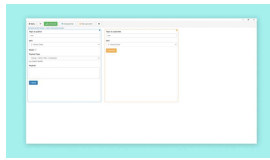
Username: the one you defined previously in the tutorial

Password: the one for the user you defined previously in the tutorial

Host: This is your Portenta X8 board IP address

Leave everything else as default and save the settings of the client. If everything is ok, you should see now that the MQTT client connected with the local

start sending messages to the MQTT broker.



MQTTBox graphical user interface (GUI).

When MQTTBox client connects to the local Mosquitto broker deployed in our Portenta X8 board, a blue "Not Connected" button should change to a green "Connected" button; also, notice that with the MQTTBox client, we are going to publish data to the `test` topic. Now, let's install Node-RED.

Installing Node-RED

Node-RED is an open-source programming tool that connects hardware with API's and online services. It is a visual tool designed for Internet of Things devices and applications, but it can also be used for other applications. The simplest form to run Node-RED with Docker is by using the following command:

```
1 docker run -it -p 1880
```

Portenta X8 board. Let's dissect the command:

`-it`: a terminal session is attached to the container so we can see what is happening with the container

`-p 1880:1880`: Node-RED local port `1880` connects to the exposed internal port `1880`

`v node_red_data:/data`: a docker named volume called `node_red_data` is mounted to the container `/data` directory. This permits any changes to the flow to persist

`--name mynodered`: a friendly local name

`nodered/node-red`: the image to base it on

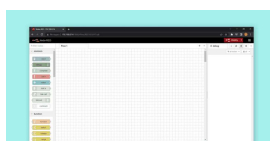
After running the command, we should see a running instance of Node-RED in the terminal. The local instance of Node-RED should be ready; let's test it! We can detach the terminal with `Ctrl-p` `Ctrl-q`; this doesn't stop the container; the container will run in the background.

Testing Node-RED

Let's browse to

`http://{your-portenta-ip}:1880`

; this will open the Node-RED desktop as shown in the image below:



Node-RED graphical user interface (GUI).

Node-RED desktop is a GUI that lets us work with Node-RED flows graphically. We can test Node-RED by connecting to the local MQTT broker we set up before using a Node-RED flow. In the **Nodes** section located in the left part of the browser, search for **network** and choose the **mqtt in** node and drop it in the workspace; we will use this node to connect to the local MQTT broker of the X8. To change the node's properties, double-click on it and define the following properties:

Server:

your-portenta-ip:1883

Action:

Subscribe to the single topic

Topic: test

QoS: 0

Output:

auto-detect (**string** or **buffer**)

Name: MQTT Broker X8

Now, search for the **change** node and drop it in the workspace; we will use this node to change the data format from the MQTT broker (string to number). To change the node's properties, double-click on it and define the following properties:

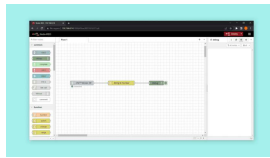
Name:

String to Number

Set: msg.payload

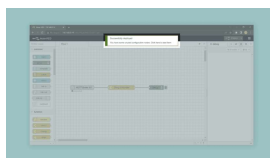
To the value:

Now, let's search for the `debug` node and drop it in the workspace; we will use this node to check if Node-RED is getting data from the `test` topic and if Node-RED is formatting the data correctly. Define the name of the node as `debug` and then connect the nodes as shown in the image below:



Node-RED flow used for testing the Portenta X8 local MQTT broker.

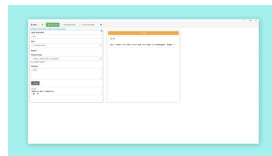
After connecting the nodes, we must deploy the Node-RED application by selecting the "Deploy" button on the browser's superior right side. We should see a "Successfully deployed" message if everything is ok, as shown in the image below:



Node-RED flow successfully deployed.

Let's use the MQTT client described before to test the MQTT broker integration with Node-RED. Just beneath the "Deploy" button, look for an **icon with a bug; Node-RED's**

can start sending messages to the local MQTT broker in the X8 and see them deployed in the debug interface of Node-RED. With the MQTT client, let's subscribe first to the `test` topic and then publish any value to the topic, as shown in the image below:



Publish data to a topic of the MQTT broker using the MQTTBox client.

We should now see data in the debug interface of Node-RED, as shown in the image below:



Debug interface of Node-RED showing data from the MQTT broker.

We can now proceed to configure InfluxDB.

Installing InfluxDB

InfluxDB is an open-source, high-performance, time series database; with InfluxDB data can be written and read in real-time, and data can be processed in

purposes or for monitoring and alerting purposes. User dashboards for visualizing and exploring data can also be set up.

The simplest form to run InfluxDB with Docker is by using the following command:

```
1 docker run --detach --
```

This command will run an InfluxDB container **locally** in our Portenta X8 board. Let's dissect the command:

`--detach`: no terminal session is attached to the container

`--name`: the container name

`-p 1880:1880`: InfluxDB local port `8086` connects to the exposed internal port `8086`

The container should now be running in the background; let's test the local instance of InfluxDB!

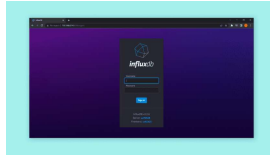
Testing InfluxDB

For testing the local instance of InfluxDB we are going to use its desktop and also Node-RED.

Let's browse to

```
http://{your-portenta-ip}:8086
```

; this will open the InfluxDB desktop as shown in the image

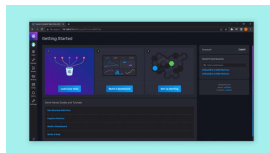


Sign in page of the InfluxDB desktop.

InfluxDB desktop is a GUI that lets us work with InfluxDB graphically.

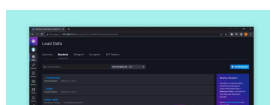
 The first time you enter the InfluxDB desktop, a username, and a password must be set up.

After setting up a username and a password, we are going to be redirected to the "Getting Started" page, as shown in the image below:



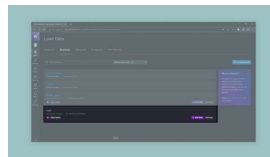
Getting started page of the InfluxDB desktop.

In this example, we will send data from the MQTT broker to InfluxDB using Node-RED; Node-RED will act as a bridge between the MQTT broker and InfluxDB. Let's go to "Data" we are going to be redirected to the "Load Data" page, as shown in the image below:



Load data page of the
InfluxDB desktop.

Select "Buckets" and click on the
"Create Bucket" button. This will
create a new database where
data from the MQTT broker will
be saved. For this example, let's
create a new bucket called
`test`; now, we should see the
bucket on the InfluxDB desktop:



Test bucket in the load
data page of the
InfluxDB desktop.

InfluxDB is now ready to receive
data; let's send it using an MQTT
client and Node-RED. We must
first set up Node-RED by
installing custom InfluxDB
nodes; click on the menu icon on
the top right corner of the
browser, then click on the
"Manage palette" option. This
will open a tab where we can see
all the palettes/nodes installed
in the Node-RED instance
running on our X8 board. Click
on the "Install" option and
search for "InfluxDB"; install the
nodes from InfluxDB. Now we
should see the InfluxDB nodes
installed in our Node-RED
instance, as shown in the image
below:



Nodes installed in the
Node-RED instance.

Now, search for the
`influxdb out` node and drop it
in the workspace; we will use
this node to send information
from the MQTT client to the
`test` bucket on InfluxDB. To
change the node's properties,
double-click on it and define the
following properties:

Name: `InfluxDB Bucket`

Organization: Your
organization name is
defined in InfluxDB

Bucket: `test`

Measurement: `counter`
(the name of the
measurement you want to
record)

Time Precision:
Milliseconds (ms)

In the **Server** option, define the
following properties:

Name: `Portenta X8`

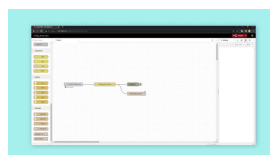
Version: `2.0`

URL:

`http://{your-portenta-
ip}:8086`

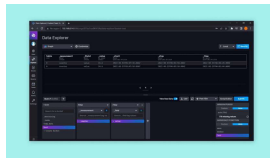
Token: the one provided
by InfluxDB

Now, connect the nodes as
shown in the image below:



Node-RED flow used
for testing the
Portenta X8 local

Let's use the MQTT client described before to test the MQTT broker integration with Node-RED and InfluxDB. Remember to first deploy the flow in Node-RED and subscribe to the `test` topic and publish any value in the MQTT client. Now, go to Data Explorer on the InfluxDB desktop; you should now see data from the MQTT client, as shown in the image below:



Visualizing data in a bucket of the Portenta X8 local InfluxDB instance.

We can now proceed to configure Grafana.

Installing Grafana

Grafana is an open-source, multi-platform data analytics and interactive data visualization solution. Grafana can connect with basically every possible data source, such as InfluxDB.

The simplest form to run Grafana with Docker is by using the following command:

```
1 docker run -d --name=g
```

This command will run a Grafana container **locally** in our Portenta X8 board. Let's dissect the command:

`-d` : no terminal session is attached to the container

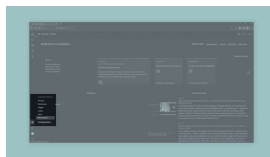
`--name` : the container name

`-p 3000:3000` : Grafana local port `3000` connects to the exposed internal port `3000`

The container should now be running in the background; let's test the local instance of Grafana!

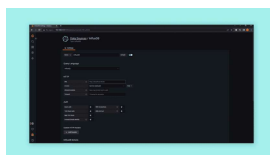
Testing Grafana

Before we can visualize data in Grafana, we must add a data source; we will add the InfluxDB bucket we created before as a data source. Select the cog icon on the side menu and then click on "Data Sources":



Adding data sources to Grafana via its GUI.

On the "Data Sources" page, select InfluxDB; this will take you to a configuration page:



In the configuration page, enter the following information related to the data bucket we created before on InfluxDB:

Name: Test Bucket

Query Language: Flux

URL:

`http://{your-portenta-ip}:8086`

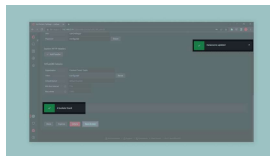
User: Your defined username in InfluxDB

Password: Your defined username password in InfluxDB

Organization: Your defined organization name in InfluxDB

Token: the one provided by InfluxDB

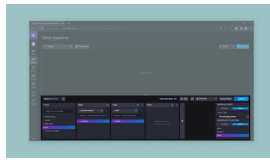
Leave the rest of the options as default. Click on the "Save & Test" button; we should see two green messages in the Grafana GUI telling us that the data source was configured correctly, as shown in the image below:



Setting up data sources in Grafana via its GUI.

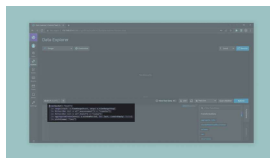
Now, let's go to the InfluxDB desktop. Go to "Data" and select the test bucket we created before; in the bucket data explorer, in the query editor, select the `test` bucket, then in `_measurement` select counter,

select last as
shown in the image below:



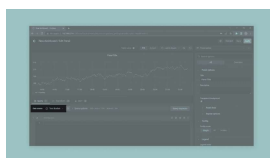
Setting up a query in
the data explorer of
the InfluxDB desktop.

Now, click on "Script Editor" and
copy the generated script by
InfluxDB; we are going to use
this script in Grafana to retrieve
information from the
bucket, as shown in the image
below:



Setting up a query in
the data explorer of
the InfluxDB desktop.

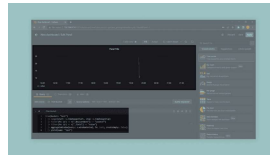
Now, in the Grafana GUI, create
a new dashboard and add a new
panel; in the configuration page,
select as "Data source" the
bucket we configured before,
:



Setting up a
dashboard in Grafana
via its GUI.

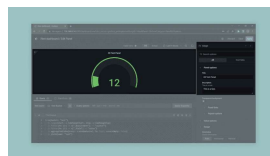
In the "Query inspector" pane

with InfluxDB. You should now see data from the tests made earlier with the MQTT client:



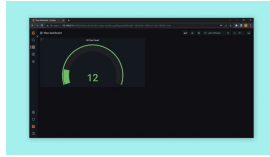
Setting up data visualization in a Grafana dashboard via its GUI.

Let's change how data is visualized. Select "Visualizations" and then search for "Gauge" and select it. Now we should visualize data as a gauge in Grafana:



Visualizing data with a gauge in a Grafana dashboard.

We can change the panel options, such as their title and description. Click on apply; we can now use the MQTT client described before to test the MQTT broker integration with Node-RED, InfluxDB, and Grafana. Remember first to deploy the flow in Node-RED, subscribe to the test topic, and publish any value in the MQTT client. Also, remember to change the dashboard time range and its refresh rate.



Configured dashboard
in Grafana.

Sending Data Using the MKR WiFi 1010 Board

Now, it is time to test our entire data-logging application. We will use an **MKR WiFi 1010**; this board will periodically send the value of a counter to the Grafana dashboard via the local MQTT broker deployed in the X8.

First, let's ensure we have the required core for the MKR WiFi 1010 installed, the **Arduino SAMD boards (32-bit Arm® Cortex®-M0+)**. Please, refer to [this guide](#) if you are not familiar with the installation of additional cores in the Arduino IDE. We also need to install the libraries we will use to send data from the MKR WiFi 1010 board to the data logging application via MQTT. Go to **Tools > Manage libraries...**, search for **ArduinoMqttClient** and **WiFiNINA**, and install the latest available version of both libraries.

Now, let's open a new sketch and create a new header file called `arduino_secrets.h` in a

separate tab in the Arduino IDE,
click the arrow symbol
underneath the Serial Monitor
symbol, then click on the "New
tab" option. In this header file,
we are going to store our Wi-Fi®
credentials:

```
1 #define SECRET_SSID "y  
2 #define SECRET_PASS "y
```

Now, let's program the following
sketch in the MKR WiFi 1010
board:

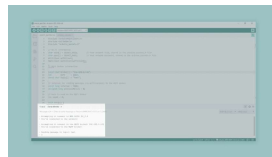
```
1 #include <ArduinoMqt  
2 #include <WiFiNINA.h  
3 #include "arduino_se  
4  
5 // Wi-Fi® informatic  
6 char ssid[] = SECRET  
7 char pass[] = SECRET  
8 WiFiClient wifiClier  
9 MqttClient mqttClier  
10  
11 // MQTT broker infor  
12  
13 const char broker[]  
14 int port  
15 const char topic[]  
16  
17 // Interval for senc  
18 const long interval  
19 unsigned long previc  
20  
21 // Data to send to t  
22 int count = 0;  
23  
24 void setup() {  
25     Serial.begin(9600)  
26     while (!Serial) {  
27         ; // Wait for se  
28     }
```

The sketch shown above connects the MKR WiFi 1010 to the local MQTT broker of the X8 and periodically sends the value of a counter to the topic `test`.

Please read [this tutorial](#) for more in-depth

i information about MQTT and the MKR WiFi 1010 board.

If everything is ok, we should see the following in the Serial monitor of the Arduino IDE :



Debug messages in the Arduino IDE 2 Serial Monitor.

Check out now the Grafana dashboard we configured earlier; we should see data coming from the MKR WiFi 1010 board.

Conclusion

In this tutorial, we went through the installation, configuration, and testing of four standard building blocks of Internet-connected data-logging devices and applications. Such blocks are an MQTT broker (Mosquitto), Node-RED, InfluxDB, and Grafana. All this runs locally in the Portenta X8 board and can

data-logging application. With this, we can further scale up your IoT development within a single Portenta X8 of the size of the palm of your hand.

Next Steps

What about controlling your house or office and making it domotic?

Do you have a sensor network that needs to be connected to the Internet to visualize sensor information?

You can use a [Portenta X8](#) board and the IoT-Quartet tutorial for developing this and more projects.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository . If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

